UNIWERSYTET WARMIŃSKO-MAZURSKI W OLSZTYNIE

Programowanie deklaratywne - paradygmaty programowania

Laboratorium 01 - Unifikacja i porównywanie termów

Unifikacja termów

Unifikacja termów w Prologu to mechanizm „uzgadniania" dwóch termów przez dobranie podstawień za zmienne tak, aby po podstawieniu oba termy stały się identyczne. Jeśli nie da się dobrać takich podstawień, unifikacja zawodzi.

Operatorem unifikacji jest `+Term1 = +Term2`, a operatorem braku unifikacji `+Term1 \= +Term2`. Poniżej przedstawiono kilka przypadków termów i ich unifikacji.

- Oba termy są stałymi.

```
?- "Adam" = "Zalewski".
false.

?- 123 = 123.
true.
```

- Jeden term jest zmienną ukonkretnioną, a drugi zmienną wolną.

```
?- adam = X.
X = adam.

?- XYZ = zalewski.
XYZ = zalewski.
```

- Jeżeli oba termy są zmiennymi wolnymi to następuje ich powiązanie.

```
?- X = Y.
X = Y.
```

```
?- XYZ = ABC., XYZ=5
XYZ = ABC, ABC = 5.
```

- Unifikacja struktur zachodzi, jeżeli posiadają ten sam funktor, tę samą liczbę argumentów oraz argumenty unifikują się parami.

```
?- f(X, 2) = f(1, Y).
X = 1,
Y = 2.

?- f(X) = g(X).          % inny funktor
false.

?- f(X) = f(X, Y).       % inna liczba argumentów
false.

?- f(X, 2) = f(1, 3).    % argumenty nie unifikują się parami
false.
```

Przykład unifikacji termów złożonych

Term1: `f(X, a, b(Y, Z))`
Term2: `f(d(a), G, b(G, c))`

Krok 1: Sprawdzenie liczby argumentów obu termów (muszą być równe)
Krok 2: Sprawdzenie nazw termów (muszą być identyczne)
Krok 3: Jeżeli **Krok 1** oraz **Krok 2** są spełnione przystępujemy do unifikacji argumentów od lewej do prawej.

Krok 3a:

Term1_1: `X`
Term2_1: `d(a)`

Za zmienną `X` możemy podstawić `d(a)`, ponieważ `d(a)` nie zawiera zmiennej `X`.

σ_1: `X = d(a)`.

Podstawienie wprowadzamy do całych termów **Term1** oraz **Term2**.

Term1σ_1: `f(d(a), a, b(Y, Z))`
Term2σ_1: `f(d(a), G, b(G, c))`

Krok 3b:

$Term1_2$: `a`
$Term2_2$: `G`

Za zmienną `G` możemy podstawić `a`, ponieważ `a` nie zawiera zmiennej `G`.

σ_2: `G = a`.

Podstawienie wprowadzamy do całych termów `Term1` oraz `Term2`.

$Term1\sigma_1\sigma_2$: `f(d(a), a, b(Y, Z))`
$Term2\sigma_1\sigma_2$: `f(d(a), a, b(a, c))`

Krok 3c:

$Term1_3$: `b(Y, Z)`
$Term2_3$: `b(a, c)`

W tym przypadku oba termy są złożone więc wracamy do sprawdzenia **Krok 1** oraz **Krok 2** w ich przypadku, a następnie wykonujemy **Krok 3**.

Krok 3d:

Za zmienną `Y` możemy podstawić `a`, ponieważ `a` nie zawiera zmiennej `Y`.

σ_3: `Y = a`.

Podstawienie wprowadzamy do całych termów `Term1` oraz `Term2`.

$Term1\sigma_1\sigma_2\sigma_3$: `f(d(a), a, b(a, Z))`
$Term2\sigma_1\sigma_2\sigma_3$: `f(d(a), a, b(a, c))`

Krok 3e:

Za zmienną `Z` możemy podstawić `c`, ponieważ `c` nie zawiera zmiennej `Z`.

σ_4: `Z = c`.

Podstawienie wprowadzamy do całych termów `Term1` oraz `Term2`.

$Term1\sigma_1\sigma_2\sigma_3\sigma_4$: `f(d(a), a, b(a, c))`
$Term2\sigma_1\sigma_2\sigma_3\sigma_4$: `f(d(a), a, b(a, c))`

Krok 4: Jeżeli unifikacja przebiegła pomyślnie i oba termy po podstawieniach są identyczne to wypisujemy MGU (najbardziej ogólne podstawienie unifikujące)

MGU(Term1, Term2) = $\sigma_1\sigma_2\sigma_3\sigma_4$ = {`X = d(a)`, `G = Y = a`, `Z = c`}

Porównywanie termów

Porównywanie termów w Prologu to sprawdzanie relacji między dwoma termami bez konieczności dopasowywania ich przez podstawienia (to robi unifikacja `=/2`). W SWI-Prolog jest dostępne kilka predykatów do porównywania, które różnią się tym co uznają za równe i czy biorą pod uwagę zmienne. Są to:

- `+Term1 == +Term2` - równość ścisła (identyczność);

```
?- X == Y.
false.

?- X = Y, X == Y.
X = Y.

?- f(X) == f(Y).
false.

?- f(X, Y) == f(X, Y).
true.
```

- `+Term1 \== +Term2` - nierówność ścisła;

```
?- X \== Y.
true.

?- X = Y, X \== Y.
false.

?- f(X) \== f(Y).
true.

?- f(X, Y) \== f(X, Y).
false.
```

- `+Term1 =@= +Term2` - równość strukturalna;

```
?- f(X, g(Y)) =@= f(A, g(B)).
true.

?- X =@= Y.
true.

?- f(X) =@= g(X).
false.
```

- `+Term1 \=@= +Term2` - nierówność strukturalna.

```
?- f(X, g(Y)) \=@= f(A, g(B)).
false.

?- X \=@= Y.
false.

?- f(X) \=@= g(X).
true.
```

Standardowy porządek termów

Standardowy porządek termów w Prologu to z góry ustalona, całkowita kolejność (total order), dzięki której każde dwa termy da się porównać — nawet jeśli zawierają zmienne.

SWI-Prolog definiuje standardowy porządek następująco:

1. Najpierw porównuje rodzaj termu: `Zmienne < Liczby < Stringi < Atomy < Termy złożone`
2. Potem porównuje wewnątrz kategorii:
   - `Zmienne`: według „adresu/wieku" (technicznie: kolejność tworzenia/adres).
   - `Liczby`: według wartości liczbowej.
   - `Stringi` i `atomy`: alfabetycznie (leksykograficznie).
   - `Termy złożone`: najpierw arność (liczba argumentów), potem nazwa funktora (alfabetycznie), a na końcu argumenty od lewej do prawej, rekurencyjnie tym samym porządkiem.

W SWI-Prologu ten porządek jest używany przez operatory:

- `+Term1 @< +Term2` - `Term1` jest przed `Term2`;

```
?- 7 @< adam.
true.

?- f(X) @< X.
false.
```

- `+Term1 @=< +Term2` - `Term1` jest przed lub jest identyczny z `Term2`;

```
?- adam @=< "adam".
false.

?- "adam" @=< "adam".
true.
```

- `+Term1 @> +Term2` - `Term1` jest po `Term2`;

```
?- adam @> X.
true.

?- "adam" @> f(X).
false.
```

- `+Term1 @>= +Term2` - `Term1` jest po lub jest identyczny z `Term2`;

```
?- f(x, y, z) @>= g(z, y, x).
false.

?- abc @>= abc.
true.
```

- `compare(Order, +Term1, +Term2)` - sprawdza porządek (`Order`) między `Term1`, a `Term2`. Order jest postaci `<`, `>` lub `=`.

```
?- compare(Order, X, Y).                % X zostało stworzone przed Y
Order = (<).

?- compare(Order, 123, 132).            % 123 jest przed 132
Order = (<).

?- compare(Order, abc, abd).            % c jest przed d
Order = (<).

?- compare(Order, "xxy", "xxz").        % y jest przed z
Order = (<).

?- compare(Order, f(X), f(X, Y)).       % porównanie liczby argumentów - 1 jest
przed 2
Order = (<).

?- compare(Order, f(X), g(X)).          % porównanie nazwy - f jest przed g
Order = (<)

?- compare(Order, f(x, y), f(x, z)).    % porównanie argumentów - y jest przed z
Order = (<).
```

Zadania

Zadania należy spakować w .zip i odesłać we wskazane przez prowadzącego miejsce.

1. W Prologu zunifikuj następujące termy:

Term1	Term2
pies	pies
kot	pies
a(b)	b
a(b, B)	a(b, c)
punkt(X, Y, Z)	punkt(3, 4, 5)
litera("a")	slowo(litera)
lubi(adam, X)	lubi(Y, ewa)
kradnie(X, Y)	kradnie(adam, cenne(zloto))

2. Do każdego operatora w `Porównywanie termów` podaj po dwa przykłady (jeden `true`, a drugi `false`).

3. Do każdego operatora w `Standardowy porządek termów` podaj po dwa przykłady (jeden `true`, a drugi `false`). Dla operatora `compare(Order, +Term1, +Term2)` podaj po jednym przykładzie kiedy `Order` to `<`, `>` lub `=`.

4. Według `Przykład unifikacji termów złożonych` rozpisz unifikację termów.

Term1	Term2
f(X, a(b, Y), c)	f(d(Z, Y), a(A, p(d)) , Z)
punkt(x(Y, d), y(d, X), z(X, Y, Z))	punkt(Z, y(d, f), z(A, p(x, y), B))